# AI ASSISTED CODING

# ASSIGNMENT-11.1

**Name: Harshitha Guda**

**H.T.NO:2303A51102**

**Task Description #1 – Stack Implementation**

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:class Stack:pass

Expected Output: A functional stack implementation with all required methods and docstrings.

```python
#generate a code to demonstrate stack and its operations and also include comments and docstrings for better understanding.
class Stack:
    """
    A class to represent a stack data structure.

    Attributes:
    stack (list): A list to store the elements of the stack.

    Methods:
    push(item): Adds an item to the top of the stack.
    pop(): Removes and returns the item at the top of the stack.
    peek(): Returns the item at the top of the stack without removing it.
    is_empty(): Checks if the stack is empty.
    size(): Returns the number of items in the stack.
    """

    def __init__(self):
        """Initialize an empty stack."""
        self.stack = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.stack.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack. Raises an error if the stack is empty."""
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot pop from an empty stack.")
        return self.stack.pop()

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises an error if the stack is empty."""
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot peek from an empty stack.")
        return self.stack[-1]

    def is_empty(self):
```

```python
    def is_empty(self):
        """Check if the stack is empty."""
        return len(self.stack) == 0

    def size(self):
        """Return the number of items in the stack."""
        return len(self.stack)
# Example usage of the Stack class
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)

    print("Top item:", stack.peek())  # Output: Top item: 3
    print("Stack size:", stack.size())  # Output: Stack size: 3

    print("Popped item:", stack.pop())  # Output: Popped item: 3
    print("Top item after pop:", stack.peek())  # Output: Top item after pop: 2
    print("Is stack empty?", stack.is_empty())  # Output: Is stack empty? False

    stack.pop()
    stack.pop()
    print("Is stack empty after popping all items?", stack.is_empty())  # Output: Is stack empty after popping all items? True
    # Uncommenting the following line will raise an error because the stack is empty
    # stack.pop()
```

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Top item: 3
Stack size: 3
Popped item: 3
Top item after pop: 2
Is stack empty? False
Is stack empty after popping all items? True
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

## Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:class Queue:

pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```python
#generate a code to demonstrate queue and its operations FIFO-based queue class with enqueue, dequeue, peek, and size methods
# and also include comments and docstrings for better understanding
class Queue:
    """
    A class to represent a queue data structure.

    Attributes:
    queue (list): A list to store the elements of the queue.

    Methods:
    enqueue(item): Adds an item to the end of the queue.
    dequeue(): Removes and returns the item at the front of the queue.
    peek(): Returns the item at the front of the queue without removing it.
    is_empty(): Checks if the queue is empty.
    size(): Returns the number of items in the queue.
    """

    def __init__(self):
        """Initialize an empty queue."""
        self.queue = []

    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.queue.append(item)

    def dequeue(self):
        """Remove and return the item at the front of the queue. Raises an error if the queue is empty."""
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot dequeue from an empty queue.")
        return self.queue.pop(0)

    def peek(self):
        """Return the item at the front of the queue without removing it. Raises an error if the queue is empty."""
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot peek from an empty queue.")
        return self.queue[0]
```

```python
    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.queue) == 0

    def size(self):
        """Return the number of items in the queue."""
        return len(self.queue)
# Example usage of the Queue class
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)

    print("Front item:", queue.peek())  # Output: Front item: 1
    print("Queue size:", queue.size())  # Output: Queue size: 3

    print("Dequeued item:", queue.dequeue())  # Output: Dequeued item: 1
    print("Front item after dequeue:", queue.peek())  # Output: Front item after dequeue: 2
    print("Is queue empty?", queue.is_empty())  # Output: Is queue empty? False

    queue.dequeue()
    queue.dequeue()
    print("Is queue empty after dequeuing all items?", queue.is_empty())  # Output: Is queue empty after dequeuing all items? True
    # Uncommenting the following line will raise an error because the queue is empty
    # queue.dequeue()
```

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Front item: 1
Queue size: 3
Dequeued item: 1
Front item after dequeue: 2
Is queue empty? False
Is queue empty after dequeuing all items? True
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

## Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:pass

class LinkedList:pass

Expected Output:A working linked list implementation with clear method documentation.

```python
#Generate code to demonstarte linked list and its operations such as insertion, deletion, traversal, and searching.
# Include comments and docstrings for better understanding.
class Node:
    """A class representing a node in a linked list."""
    def __init__(self, data):
        self.data = data  # The data stored in the node
        self.next = None  # Pointer to the next node in the list
class LinkedList:
    """A class representing a linked list."""
    def __init__(self):
        self.head = None  # The head of the linked list
    def insert(self, data):
        """Insert a new node with the given data at the end of the list."""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node  # If the list is empty, set the new node as the head
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next  # Traverse to the end of the list
        last_node.next = new_node  # Link the new node to the end of the list
    def delete(self, key):
        """Delete the first node with the given key (data)."""
        current_node = self.head
        previous_node = None
        while current_node and current_node.data != key:
            previous_node = current_node
            current_node = current_node.next  # Traverse the list to find the node to delete
        if current_node is None:
            return  # Key not found in the list
        if previous_node is None:
            self.head = current_node.next  # If the head is to be deleted, update the head pointer
        else:
            previous_node.next = current_node.next  # Bypass the node to be deleted
```

```python
    def traverse(self):
        """Traverse the linked list and print each node's data."""
        current_node = self.head
        while current_node:
            print(current_node.data)  # Print the data of each node
            current_node = current_node.next  # Move to the next node
    def search(self, key):
        """Search for a node with the given key (data) and return True if found, else False."""
        current_node = self.head
        while current_node:
            if current_node.data == key:
                return True  # Key found in the list
            current_node = current_node.next  # Move to the next node
        return False  # Key not found in the list
# Example usage of the LinkedList class
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)
    print("Linked List after insertion:")
    linked_list.traverse()  # Output: 10, 20, 30
    print("Searching for 20:", linked_list.search(20))  # Output: True
    print("Deleting 20 from the list.")
    linked_list.delete(20)
    print("Linked List after deletion:")
    linked_list.traverse()  # Output: 10, 30
    print("Searching for 20:", linked_list.search(20))  # Output: False
```

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Linked List after insertion:
10
20
30
Searching for 20: True
Deleting 20 from the list.
Linked List after deletion:
10
30
Searching for 20: False
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

## Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:class BST:pass

Expected Output: BST implementation with recursive insert and traversal methods.

```python
# generate a code for BST implementation with recursive insert and traversal methods
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        if self.root is None:
            self.root = Node(key)
        else:
            self._insert_recursive(self.root, key)

    def _insert_recursive(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = Node(key)
            else:
                self._insert_recursive(node.left, key)
        else:
            if node.right is None:
                node.right = Node(key)
            else:
                self._insert_recursive(node.right, key)

    def inorder_traversal(self):
        return self._inorder_recursive(self.root)
```

```python
    def _inorder_recursive(self, node):
        res = []
        if node:
            res = self._inorder_recursive(node.left)
            res.append(node.val)
            res = res + self._inorder_recursive(node.right)
        return res

    def preorder_traversal(self):
        return self._preorder_recursive(self.root)

    def _preorder_recursive(self, node):
        res = []
        if node:
            res.append(node.val)
            res = res + self._preorder_recursive(node.left)
            res = res + self._preorder_recursive(node.right)
        return res

    def postorder_traversal(self):
        return self._postorder_recursive(self.root)

    def _postorder_recursive(self, node):
        res = []
        if node:
            res = self._postorder_recursive(node.left)
            res = res + self._postorder_recursive(node.right)
            res.append(node.val)
        return res
```

```python
# Example usage
if __name__ == "__main__":
    bst = BST()
    bst.insert(10)
    bst.insert(5)
    bst.insert(15)
    bst.insert(3)
    bst.insert(7)
    bst.insert(12)
    bst.insert(18)

    print("Inorder Traversal:", bst.inorder_traversal())
    print("Preorder Traversal:", bst.preorder_traversal())
    print("Postorder Traversal:", bst.postorder_traversal())
```

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Inorder Traversal: [3, 5, 7, 10, 12, 15, 18]
Preorder Traversal: [10, 5, 3, 7, 15, 12, 18]
Postorder Traversal: [3, 7, 5, 12, 18, 15, 10]
```

**Task Description #5 – Hash Table**

Task: Use AI to implement a hash table with basic insert, search, and

delete methods.

Sample Input Code:

class HashTable:pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

```python
#genarate a code for to implement a hash table with basic insert, search,
# and delete methods include Collision handling using chaining, with well-commented methods.
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [None] * size

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash(key)
        if self.table[index] is None:
            self.table[index] = Node(key, value)
        else:
            current = self.table[index]
            while current:
                if current.key == key:
                    current.value = value  # Update existing key
                    return
                if current.next is None:
                    current.next = Node(key, value)
                    return
                current = current.next
```

```python
    def search(self, key):
        index = self._hash(key)
        current = self.table[index]
        while current:
            if current.key == key:
                return current.value
            current = current.next
        raise KeyError(key)

    def delete(self, key):
        index = self._hash(key)
        current = self.table[index]
        if not current:
            raise KeyError(key)
        if current.key == key:
            self.table[index] = current.next
            return
        while current.next:
            if current.next.key == key:
                current.next = current.next.next
                return
            current = current.next
        raise KeyError(key)
```

```python
# Example usage:
if __name__ == "__main__":
    ht = HashTable()
    ht.insert("name", "Alice")
    ht.insert("age", 30)
    print(ht.search("name"))  # Output: Alice
    print(ht.search("age"))   # Output: 30
    ht.delete("name")
    try:
        print(ht.search("name"))  # This will raise a KeyError
    except KeyError:
        print("Key 'name' not found")
    print(ht.search("age"))   # Output: 30
```

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Alice
30
Key 'name' not found
30
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

## Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:pass

Expected Output:

• Graph with methods to add vertices, add edges, and display connections.

```python
#generate a code to to implement a graph using an adjacency list and implement methods to add vertices, add edges, and display connections
class Graph:
    def __init__(self):
        self.graph = {}

    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)

    def display_connections(self):
        for vertex in self.graph:
            connections = ', '.join(self.graph[vertex])
            print(f"{vertex} --> {connections}")
# Example usage
if __name__ == "__main__":
    g = Graph()
    g.add_vertex("A")
    g.add_vertex("B")
    g.add_vertex("C")
    g.add_edge("A", "B")
    g.add_edge("A", "C")
    g.add_edge("B", "C")
    g.display_connections()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
A --> B, C
B --> A, C
C --> A, B
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

## Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heap q module.

Sample Input Code:

class PriorityQueue:pass

Expected Output:

• Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```python
#generate code to implement a priority queue using Python's heap queue module.
#and implement enqueue (priority), dequeue (highest priority), and display methods.
import heapq

class PriorityQueue:
    def __init__(self):
        self._queue = []
        self._index = 0
    def enqueue(self, item, priority):
        heapq.heappush(self._queue, (priority, self._index, item))
        self._index += 1
    def dequeue(self):
        return heapq.heappop(self._queue)[-1]
    def display(self):
        for priority, index, item in self._queue:
            print(f"Item: {item}, Priority: {priority}")
# Example usage
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue("Task 1", priority=2)
    pq.enqueue("Task 2", priority=1)
    pq.enqueue("Task 3", priority=3)
    print("Priority Queue:")
    pq.display()
    print("\nDequeueing highest priority item:")
    print(pq.dequeue())
    print("\nPriority Queue after dequeue:")
    pq.display()
```

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Priority Queue:
Item: Task 2, Priority: 1
Item: Task 1, Priority: 2
Item: Task 3, Priority: 3

Dequeueing highest priority item:
Task 2

Priority Queue after dequeue:
Item: Task 1, Priority: 2
Item: Task 3, Priority: 3
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

## Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using

collections.deque.

Sample Input Code:class DequeDS:pass

Expected Output:Insert and remove from both ends with docstrings.

```python
#gnerate a code to implement a double-ended queue using collections.
#Insert and remove from both ends with docstrings.
from collections import deque
class DoubleEndedQueue:
    """
    A double-ended queue (deque) implementation using collections.deque.
    This class allows insertion and removal of elements from both ends of the queue.
    """
    def __init__(self):
        """Initialize an empty deque."""
        self.deque = deque()
    def insert_front(self, item):
        """
        Insert an item at the front of the deque.

        Parameters:
        item: The item to be inserted at the front of the deque.
        """
        self.deque.appendleft(item)
    def insert_rear(self, item):
        """
        Insert an item at the rear of the deque.

        Parameters:
        item: The item to be inserted at the rear of the deque.
        """
        self.deque.append(item)
```

```python
    def insert_rear(self, item):
        """
        Insert an item at the rear of the deque.

        Parameters:
        item: The item to be inserted at the rear of the deque.
        """
        self.deque.append(item)
    def remove_front(self):
        """
        Remove and return an item from the front of the deque.

        Returns:
        The item removed from the front of the deque. If the deque is empty, returns None.
        """
        if self.deque:
            return self.deque.popleft()
        else:
            return None
    def remove_rear(self):
        """
        Remove and return an item from the rear of the deque.

        Returns:
        The item removed from the rear of the deque. If the deque is empty, returns None.
        """
        if self.deque:
            return self.deque.pop()
        else:
            return None
    def __str__(self):
        """Return a string representation of the deque."""
        return str(list(self.deque))
```

```python
# Example usage:
if __name__ == "__main__":
    deque_instance = DoubleEndedQueue()
    deque_instance.insert_rear(1)
    deque_instance.insert_rear(2)
    deque_instance.insert_front(0)
    print("Deque after insertions:", deque_instance)

    print("Removed from front:", deque_instance.remove_front())
    print("Removed from rear:", deque_instance.remove_rear())
    print("Deque after removals:", deque_instance)
```

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Deque after insertions: [0, 1, 2]
Removed from front: 0
Removed from rear: 2
Deque after removals: [1]
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

**Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure**

Scenario:

Your college wants to develop a Campus Resource Management System

that handles:

1. Student Attendance Tracking – Daily log of studentsentering/exiting the campus.

2. Event Registration System – Manage participants in events withmquick search and removal.

3. Library Book Borrowing – Keep track of available books and their due dates.

4. Bus Scheduling System – Maintain bus routes and stop connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

• For each feature, select the most appropriate data structure fromthe list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

```python
#generate a code to implement Cafeteria Order Queue using queue data structutre
class CafeteriaOrderQueue:
    def __init__(self):
        self.queue = []
    def enqueue(self, order):
        self.queue.append(order)
        print(f"Order '{order}' added to the queue.")
    def dequeue(self):
        if not self.is_empty():
            order = self.queue.pop(0)
            print(f"Order '{order}' removed from the queue.")
            return order
        else:
            print("The queue is empty. No orders to dequeue.")
            return None
    def peek(self):
        if not self.is_empty():
            print(f"The next order in the queue is: '{self.queue[0]}'.")
            return self.queue[0]
        else:
            print("The queue is empty. No orders to peek.")
            return None
    def is_empty(self):
        return len(self.queue) == 0
    def size(self):
        print(f"The number of orders in the queue is: {len(self.queue)}.")
        return len(self.queue)
```

```python
# Example usage
if __name__ == "__main__":
    cafeteria_queue = CafeteriaOrderQueue()
    cafeteria_queue.enqueue("Burger")
    cafeteria_queue.enqueue("Pizza")
    cafeteria_queue.enqueue("Salad")
    cafeteria_queue.peek()
    cafeteria_queue.size()
    cafeteria_queue.dequeue()
    cafeteria_queue.peek()
    cafeteria_queue.size()
    cafeteria_queue.dequeue()
    cafeteria_queue.dequeue()
    cafeteria_queue.dequeue()  # Attempt to dequeue from an empty queue
    cafeteria_queue.peek()  # Attempt to peek into an empty queue
    cafeteria_queue.size()  # Check size of the queue when it's empty
```

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Order 'Burger' added to the queue.
Order 'Pizza' added to the queue.
Order 'Salad' added to the queue.
The next order in the queue is: 'Burger'.
The number of orders in the queue is: 3.
Order 'Burger' removed from the queue.
The next order in the queue is: 'Pizza'.
The number of orders in the queue is: 2.
Order 'Pizza' removed from the queue.
Order 'Salad' removed from the queue.
The queue is empty. No orders to dequeue.
The queue is empty. No orders to peek.
The number of orders in the queue is: 0.
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

**Justification:**

The **queue** data structure is the best fit for a cafeteria order system because it naturally models the way people line up to be served. A queue operates on the **FIFO (First-In-First-Out)** principle, meaning the first student to arrive is the first to be served, and each subsequent student is served in the order they joined the line. This ensures fairness, simplicity, and efficiency in managing the serving process without extra overhead.

| Feature | Data Structure | Justification |
|---|---|---|
| Serve students in the order they arrive | **Queue** | Follows **FIFO (First-In-First-Out)**, ensuring students are served in exact arrival order with fairness and simplicity. |
| Allow a student to leave the line | **Linked List** | Enables efficient removal from the middle without shifting all elements. |
| Handle urgent cases | **Priority Queue** | Serves high-priority students first, regardless of arrival order. |
| Quick lookup by ID | **Hash Table** | Provides constant-time average lookup for student records. |
| Serve from both ends | **Deque** | Supports insertion/removal from both front and back. |

| Feature | Data Structure | Justification |
|---|---|---|
| Maintain sorted records | **BST** | Efficiently stores and retrieves records in sorted order. |
| Represent group arrivals | **Graph** | Models relationships like friendships or linked arrivals. |
| Track history of served students | **Stack** | LIFO structure allows undoing the last serving action. |

**Task Description #10: Smart Hospital Management System – Data Structure Selection**

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.

3. Medical Records Storage – Fast retrieval of patient details using ID.

4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

```python
#Generate a code to implement Emergency Case Handling –
# Critical patients must be treated first using priority queue data structure with comments and docstrings.
class Patient:
    """
    A class to represent a patient in the emergency case handling system.

    Attributes:
    name (str): The name of the patient.
    severity (int): The severity level of the patient's condition (higher is more severe).
    """

    def __init__(self, name, severity):
        """
        Initializes a new patient with a name and severity level.

        Parameters:
        name (str): The name of the patient.
        severity (int): The severity level of the patient's condition.
        """
        self.name = name
        self.severity = severity

    def __lt__(self, other):
        """
        Defines the less than operator for comparing patients based on severity.

        Parameters:
        other (Patient): Another patient to compare with.

        Returns:
        bool: True if this patient's severity is less than the other patient's severity, False otherwise.
        """
        return self.severity < other.severity
```

```python
class EmergencyCaseHandling:
    """
    A class to manage the emergency case handling system using a priority queue.

    Attributes:
    patients (list): A list to store the patients in the priority queue.
    """

    def __init__(self):
        """
        Initializes an empty priority queue for patients.
        """
        self.patients = []

    def add_patient(self, patient):
        """
        Adds a patient to the priority queue.

        Parameters:
        patient (Patient): The patient to be added to the queue.
        """
        self.patients.append(patient)
        self.patients.sort(reverse=True)  # Sort patients by severity in descending order

    def treat_patient(self):
        """
        Treats the most critical patient (the one with the highest severity).

        Returns:
        Patient: The patient that was treated, or None if there are no patients in the queue.
        """
        if not self.patients:
            print("No patients to treat.")
            return None
        return self.patients.pop(0)  # Remove and return the most critical patient
```

```python
# Example usage
if __name__ == "__main__":
    emergency_system = EmergencyCaseHandling()

    # Adding patients to the system
    emergency_system.add_patient(Patient("Alice", 5))
    emergency_system.add_patient(Patient("Bob", 3))
    emergency_system.add_patient(Patient("Charlie", 8))

    # Treating patients
    treated_patient = emergency_system.treat_patient()
    if treated_patient:
        print(f"Treated patient: {treated_patient.name} with severity {treated_patient.severity}")

    treated_patient = emergency_system.treat_patient()
    if treated_patient:
        print(f"Treated patient: {treated_patient.name} with severity {treated_patient.severity}")

    treated_patient = emergency_system.treat_patient()
    if treated_patient:
        print(f"Treated patient: {treated_patient.name} with severity {treated_patient.severity}")

    # Attempting to treat a patient when the queue is empty
    treated_patient = emergency_system.treat_patient()
    if treated_patient:
        print(f"Treated patient: {treated_patient.name} with severity {treated_patient.severity}")
```

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Treated patient: Charlie with severity 8
Treated patient: Alice with severity 5
Treated patient: Bob with severity 3
No patients to treat.
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

| Feature | Data Structure | Justification |
|---|---|---|
| Patient Check-In System | **Queue** | Patients are treated in the order they arrive, which follows the First-In-First-Out (FIFO) principle. A queue ensures fairness and maintains arrival order. |
| Emergency Case Handling | **Priority Queue** | Critical patients must be treated first regardless of arrival time. A priority queue allows assigning higher priority to emergencies so they are dequeued before regular cases. |
| Medical Records Storage | **Hash Table** | Patient details can be retrieved quickly using their unique ID. Hash tables provide constant-time average lookup, making them ideal for fast record access. |
| Doctor Appointment Scheduling | **Binary Search Tree (BST)** | Appointments need to be sorted by time. A BST maintains sorted order and allows efficient insertion, deletion, and retrieval of appointments. |
| Hospital Room Navigation | **Graph** | Wards and rooms are interconnected like nodes and edges. A graph structure models navigation paths, enabling shortest path algorithms for efficient routing. |

**Justification:**

Patients are added with a **priority number** (lower = more critical). The **heapq** module ensures the patient with the lowest priority number is treated first. The system maintains fairness while prioritizing emergencies.

**Task Description #11: Smart City Traffic Control System**

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots. Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

```python
#Generate a code to implement Vehicle Registration Lookup – Instant access to vehicle details.
#Using Hashtables to store and retrieve vehicle information based on registration numbers.
#With comments and doctsrings for better understanding.
class VehicleRegistrationLookup:
    """
    A class to implement Vehicle Registration Lookup using Hashtables.

    This class allows you to store and retrieve vehicle information based on registration numbers.
    """

    def __init__(self):
        """Initialize the hashtable to store vehicle information."""
        self.vehicle_info = {}

    def add_vehicle(self, registration_number, details):
        """
        Add a vehicle's details to the hashtable.

        Parameters:
        registration_number (str): The registration number of the vehicle.
        details (dict): A dictionary containing the vehicle's details (e.g., make, model, year).

        Returns:
        None
        """
        self.vehicle_info[registration_number] = details
```

```python
    def lookup_vehicle(self, registration_number):
        """
        Retrieve a vehicle's details based on its registration number.

        Parameters:
        registration_number (str): The registration number of the vehicle to look up.

        Returns:
        dict: A dictionary containing the vehicle's details if found, otherwise None.
        """
        return self.vehicle_info.get(registration_number, None)
# Example usage:
if __name__ == "__main__":
    # Create an instance of VehicleRegistrationLookup
    vehicle_lookup = VehicleRegistrationLookup()

    # Add some vehicles to the lookup
    vehicle_lookup.add_vehicle("ABC123", {"make": "Toyota", "model": "Corolla", "year": 2020})
    vehicle_lookup.add_vehicle("XYZ789", {"make": "Honda", "model": "Civic", "year": 2019})

    # Lookup a vehicle by registration number
    print(vehicle_lookup.lookup_vehicle("ABC123"))  # Output: {'make': 'Toyota', 'model': 'Corolla', 'year': 2020}
    print(vehicle_lookup.lookup_vehicle("XYZ789"))  # Output: {'make': 'Honda', 'model': 'Civic', 'year': 2019}
    print(vehicle_lookup.lookup_vehicle("DEF456"))  # Output: None
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
{'make': 'Toyota', 'model': 'Corolla', 'year': 2020}
{'make': 'Honda', 'model': 'Civic', 'year': 2019}
None
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

| **Feature** | **Data Structure** | **Justification** |
|---|---|---|
| Traffic Signal Queue | **Queue** | Vehicles at a signal follow the First-In-First-Out (FIFO) principle. A queue ensures cars are released in the order they arrive. |

| Feature | Data Structure | Justification |
|---|---|---|
| Emergency Vehicle Priority Handling | **Priority Queue** | Ambulances and fire trucks must be prioritized over regular vehicles. A priority queue allows assigning higher priority so they move first regardless of arrival order. |
| Vehicle Registration Lookup | **Hash Table** | Vehicle details can be retrieved instantly using registration numbers. Hash tables provide constant-time average lookup, making them ideal for fast access. |
| Road Network Mapping | **Graph** | Roads and intersections are interconnected like nodes and edges. A graph structure models the network, enabling shortest path algorithms for efficient routing. |
| Parking Slot Availability | **Deque** | Parking slots can be tracked as available or occupied, with insertion/removal at both ends. A deque allows efficient updates when cars enter or leave from either side of the parking lot. |

**Justification:**

Vehicle registration numbers act as unique keys, making them ideal for direct mapping in a hash table.Hash tables provide **constant-time average lookup**, ensuring instant access to vehicle details.This efficiency is crucial in traffic systems where delays could cause congestion or hinder enforcement.Unlike sequential structures, hash tables avoid linear searches, making them scalable for millions of records.

**Task Description #12: Smart E-Commerce Platform – Data Structure Challenge**

An e-commerce company wants to build a Smart Online Shopping Systemwith:

1. Shopping Cart Management – Add and remove products dynamically.

2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

• For each feature, select the most appropriate data structure from list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output: A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

```python
from collections import defaultdict, deque
import heapq
"""
Delivery Route Planning using Graph Data Structure

This module implements a delivery route planner using graph algorithms
to find optimal routes between delivery locations.
"""
class DeliveryGraph:
    """
    A graph-based delivery route planner.
    Represents locations as nodes and distances/travel times as weighted edges.
    """
    def __init__(self):
        """Initialize the delivery graph."""
        self.graph = defaultdict(list)
        self.locations = {}
    def add_location(self, location_id, name, coordinates):
        """
        Add a delivery location to the graph.
        Args:
            location_id: Unique identifier for the location
            name: Name of the location
            coordinates: Tuple of (latitude, longitude)
        """
        self.locations[location_id] = {"name": name, "coordinates": coordinates}
```

```python
    def add_route(self, from_location, to_location, distance):
        """
        Add a bidirectional route between two locations.
        Args:
            from_location: Starting location ID
            to_location: Destination location ID
            distance: Distance/time between locations
        """
        self.graph[from_location].append((to_location, distance))
        self.graph[to_location].append((from_location, distance))
    def dijkstra_shortest_path(self, start, end):
        """
        Find the shortest path between two locations using Dijkstra's algorithm.
        Args:
            start: Starting location ID
            end: Destination location ID
        Returns:
            Tuple of (shortest_distance, path) or (float('inf'), []) if no path
        """
        distances = {location: float('inf') for location in self.locations}
        distances[start] = 0
        previous = {location: None for location in self.locations}
        pq = [(0, start)]
        visited = set()
        while pq:
            current_dist, current = heapq.heappop(pq)
            if current in visited:
                continue
            visited.add(current)
            if current == end:
                # Reconstruct path
```

```python
            # Reconstruct path
            path = []
            node = end
            while node is not None:
                path.append(node)
                node = previous[node]
            return current_dist, path[::-1]
        for neighbor, distance in self.graph[current]:
            new_dist = current_dist + distance
            if new_dist < distances[neighbor]:
                distances[neighbor] = new_dist
                previous[neighbor] = current
                heapq.heappush(pq, (new_dist, neighbor))
    return float('inf'), []
def get_route_details(self, route):
    """
    Get human-readable details for a route.
    Args:
        route: List of location IDs
    Returns:
        List of location names in the route
    """
    return [self.locations[loc]["name"] for loc in route]
```

```python
# Example usage
if __name__ == "__main__":
    # Create delivery planner
    planner = DeliveryGraph()
    # Add delivery locations
    planner.add_location(1, "Warehouse", (0, 0))
    planner.add_location(2, "Store A", (2, 3))
    planner.add_location(3, "Store B", (5, 1))
    planner.add_location(4, "Store C", (3, 5))
    # Add routes with distances
    planner.add_route(1, 2, 10)
    planner.add_route(1, 3, 15)
    planner.add_route(2, 3, 8)
    planner.add_route(2, 4, 12)
    planner.add_route(3, 4, 9)
    # Find shortest path
    distance, path = planner.dijkstra_shortest_path(1, 4)
    route_names = planner.get_route_details(path)
    print(f"Shortest distance: {distance}")
    print(f"Delivery route: {' -> '.join(route_names)}")
```

| Feature | Data Structure | Justification |
|---|---|---|
| Shopping Cart Management | **Linked List** | Products in a cart need dynamic addition and removal. A linked list allows efficient insertions and deletions without shifting elements, making it ideal for managing cart items. |
| Order Processing System | **Queue** | Orders must be processed in the order they are placed, following the First-In-First-Out (FIFO) principle. A queue ensures fairness and sequential handling of customer orders. |
| Top-Selling Products Tracker | **Priority Queue** | Products should be ranked by sales count, with the highest-selling products accessed first. A priority queue efficiently maintains this ranking by priority values. |
| Product Search Engine | **Hash Table** | Products are identified by unique IDs. A hash table provides constant-time average lookup, ensuring fast retrieval of product details. |

| Feature | Data Structure | Justification |
|---|---|---|
| Delivery Route Planning | **Graph** | Warehouses and delivery locations form a network of nodes and edges. A graph models these connections and supports shortest path algorithms for efficient delivery routing. |

**Justification:**

Delivery routes involve warehouses, hubs, and customer locations that are interconnected, much like nodes and edges in a graph.A graph structure allows modeling of these connections logically, capturing both direct and indirect paths.Algorithms such as Dijkstra's or BFS can then be applied to find the shortest or most efficient delivery route.This ensures optimized logistics, reducing delivery time and cost while handling complex city-wide or regional networks effectively.

**Task #13: Smart Logistics & Warehouse Management**

Scenario:

A logistics company wants to implement:

1. Shipment Processing

2. Urgent Shipment Priority

3. Product Lookup by Code

4. Warehouse Network Connectivity

5. Inventory Add/Remove Operations

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

```python
from collections import deque
"""
Inventory Management System using Deque Data Structure

This module implements an inventory system that uses a deque (double-ended queue)
to manage product addition and removal operations efficiently.
"""
class InventoryManager:
    """
    A class to manage inventory operations using deque data structure.
    Attributes:
        inventory (deque): A deque to store inventory items
    """
    def __init__(self):
        """Initialize an empty inventory deque."""
        self.inventory = deque()
    def add_item(self, item):
        """
        Add an item to the inventory (append to the right).

        Args:
            item (str): The item to add to inventory
        """
        self.inventory.append(item)
        print(f"Added '{item}' to inventory")
    def remove_item_fifo(self):
        """
        Remove an item from inventory (FIFO - First In First Out).
        Returns:
            str: The removed item, or None if inventory is empty
        """
        if self.inventory:
            item = self.inventory.popleft()
            print(f"Removed '{item}' from inventory (FIFO)")
            return item
```

```python
        else:
            print("Inventory is empty!")
            return None
    def remove_item_lifo(self):
        """
        Remove an item from inventory (LIFO - Last In First Out).
        Returns:
            str: The removed item, or None if inventory is empty
        """
        if self.inventory:
            item = self.inventory.pop()
            print(f"Removed '{item}' from inventory (LIFO)")
            return item
        else:
            print("Inventory is empty!")
            return None
    def display_inventory(self):
        """Display all items currently in the inventory."""
        if self.inventory:
            print("Current Inventory:", list(self.inventory))
        else:
            print("Inventory is empty!")
# Demo usage
if __name__ == "__main__":
    manager = InventoryManager()
    # Add items
    manager.add_item("Laptop")
    manager.add_item("Mouse")
    manager.add_item("Keyboard")
    # Display inventory
    manager.display_inventory()
    # Remove item (FIFO)
    manager.remove_item_fifo()
```

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Added 'Laptop' to inventory
Added 'Mouse' to inventory
Added 'Keyboard' to inventory
Current Inventory: ['Laptop', 'Mouse', 'Keyboard']
Removed 'Laptop' from inventory (FIFO)
Removed 'Keyboard' from inventory (LIFO)
Current Inventory: ['Mouse']
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

| Feature | Data Structure | Justification |
|---|---|---|
| Shipment Processing | **Queue** | Shipments are processed in the order they arrive, following the First-In-First-Out (FIFO) principle. A queue ensures fairness and sequential handling of shipments. |
| Urgent Shipment Priority | **Priority Queue** | Urgent shipments must be prioritized over regular ones. A priority queue allows assigning higher priority so critical shipments are dispatched first. |
| Product Lookup by Code | **Hash Table** | Products are identified by unique codes. A hash table provides constant-time average lookup, ensuring fast retrieval of product details. |
| Warehouse Network Connectivity | **Graph** | Warehouses and distribution centers form a network of nodes and edges. A graph models these connections and supports shortest path algorithms for efficient routing. |
| Inventory Add/Remove Operations | **Deque** | Inventory requires flexible addition and removal from both ends (e.g., loading/unloading). A deque supports efficient operations at both ends, making it ideal for inventory management. |

**Justification:**

Inventory systems often require adding or removing items from both ends, such as loading new stock or dispatching old stock.A deque supports efficient insertions and deletions at both the front and rear, unlike stacks or queues which restrict operations.This flexibility makes it ideal for warehouse inventory management where operations can occur dynamically at either end.

**Task Description #14: Smart Banking Management System – Data Structure Challenge**

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.

2. Loan Approval Priority Handling – High-value or urgent loans processed first.

3. Account Information Lookup – Fast retrieval of customer details using Account Number.

4. Transaction History Management – Maintain chronological transaction records.

5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

```python
"""
Transaction History Management using LinkedList Data Structure
Manages financial transactions with FIFO (First In First Out) behavior
"""
class Node:
    """Represents a single transaction node in the linked list"""
    def __init__(self, transaction_id, amount, transaction_type, timestamp):
        """
        Initialize a transaction node

        Args:
            transaction_id: Unique identifier for the transaction
            amount: Transaction amount
            transaction_type: Type of transaction (deposit/withdrawal)
            timestamp: Time of transaction
        """
        self.transaction_id = transaction_id
        self.amount = amount
        self.transaction_type = transaction_type
        self.timestamp = timestamp
        self.next = None
class TransactionHistory:
    """Manages transaction history using a linked list"""
    def __init__(self):
        """Initialize empty transaction history"""
        self.head = None
        self.size = 0
    def add_transaction(self, transaction_id, amount, transaction_type, timestamp):
        """Add a new transaction to the history"""
        new_node = Node(transaction_id, amount, transaction_type, timestamp)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
```

```python
            current.next = new_node
        self.size += 1
    def display_history(self):
        """Display all transactions"""
        if not self.head:
            print("No transactions found")
            return
        current = self.head
        while current:
            print(f"ID: {current.transaction_id} | Type: {current.transaction_type} | "
                  f"Amount: ${current.amount} | Time: {current.timestamp}")
            current = current.next
    def get_total_balance(self):
        """Calculate total balance from all transactions"""
        total = 0
        current = self.head
        while current:
            if current.transaction_type == "deposit":
                total += current.amount
            else:
                total -= current.amount
            current = current.next
        return total
    def search_transaction(self, transaction_id):
        """Search for a specific transaction by ID"""
        current = self.head
        while current:
            if current.transaction_id == transaction_id:
                return current
            current = current.next
        return None
```

```python
# Example usage
if __name__ == "__main__":
    history = TransactionHistory()
    # Add transactions
    history.add_transaction(1, 500, "deposit", "2024-01-15 10:30")
    history.add_transaction(2, 200, "withdrawal", "2024-01-15 11:15")
    history.add_transaction(3, 1000, "deposit", "2024-01-15 14:45")
    # Display all transactions
    print("=== Transaction History ===")
    history.display_history()
    # Show balance
    print(f"\nTotal Balance: ${history.get_total_balance()}")
    # Search transaction
    transaction = history.search_transaction(2)
    if transaction:
        print(f"\nFound: {transaction.transaction_type} of ${transaction.amount}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
=== Transaction History ===
ID: 1 | Type: deposit | Amount: $500 | Time: 2024-01-15 10:30
ID: 2 | Type: withdrawal | Amount: $200 | Time: 2024-01-15 11:15
ID: 3 | Type: deposit | Amount: $1000 | Time: 2024-01-15 14:45

Total Balance: $1300

Found: withdrawal of $200
PS C:\Users\gudah\OneDrive\Documents\AIAC>
```

| Feature | Data Structure | Justification |
|---|---|---|
| Customer Service Token System | **Queue** | Customers are served in the order they arrive, following the First-In-First-Out (FIFO) principle. A queue ensures fairness and sequential handling of service tokens. |

| Feature | Data Structure | Justification |
|---|---|---|
| Loan Approval Priority Handling | **Priority Queue** | High-value or urgent loans must be processed first. A priority queue allows assigning higher priority so critical loan requests are handled before regular ones. |
| Account Information Lookup | **Hash Table** | Accounts are identified by unique account numbers. A hash table provides constant-time average lookup, ensuring fast retrieval of customer details. |
| Transaction History Management | **Linked List** | Transactions must be maintained in chronological order. A linked list allows efficient sequential storage and traversal of transaction records. |
| ATM Network Connectivity | **Graph** | ATMs across cities form a network of nodes and edges. A graph models these connections and supports shortest path algorithms for efficient ATM routing and connectivity. |

**Justification:**

Transaction records must be stored in chronological order, and a linked list naturally maintains sequential data.Insertions at the end (new transactions) are efficient without needing to shift existing elements.Traversal through the list allows easy review of past transactions in the exact order they occurred.

**Task Description #15: Online Learning Management System – Data Structure Selection**

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.

2. Assignment Submission Queue – Submissions processed in order of upload.

3. Top Performer Ranking – Rank students based on scores.

4. Course Prerequisite Structure – Represent subject dependencies.

5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

• Select the most appropriate data structure for each feature from the given list.

• Justify your selection in 2–3 sentences per feature.

• Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

• Feature → Data Structure → Justification table.

• Functional Python program with proper documentation.

```python
#Generate a code to implement Undo Feature in Online Editor using stack data structure.
#With comments and docstrings
class Stack:
    """A simple implementation of a stack data structure."""

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack.
        Raises an exception if the stack is empty."""
        if not self.is_empty():
            return self.items.pop()
        else:
            raise IndexError("Stack is empty")

    def peek(self):
        """Return the item at the top of the stack without removing it.
        Raises an exception if the stack is empty."""
        if not self.is_empty():
            return self.items[-1]
        else:
            raise IndexError("Stack is empty")

    def is_empty(self):
        """Check if the stack is empty."""
        return len(self.items) == 0

    def size(self):
        """Return the number of items in the stack."""
        return len(self.items)
```

```python
class OnlineEditor:
    """A simple online editor that supports undo functionality using a stack."""

    def __init__(self):
        """Initialize the online editor with an empty content and an undo stack."""
        self.content = ""
        self.undo_stack = Stack()

    def type(self, text):
        """Simulate typing text into the editor.
        The current content is saved to the undo stack before updating."""
        self.undo_stack.push(self.content)
        self.content += text

    def undo(self):
        """Undo the last action by popping the previous content from the undo stack."""
        if not self.undo_stack.is_empty():
            self.content = self.undo_stack.pop()
        else:
            print("Nothing to undo.")

    def get_content(self):
        """Return the current content of the editor."""
        return self.content
```

```python
# Example usage
if __name__ == "__main__":
    editor = OnlineEditor()
    editor.type("Hello, ")
    editor.type("world!")
    print("Current content:", editor.get_content())  # Output: Hello, world!

    editor.undo()
    print("After undo:", editor.get_content())  # Output: Hello,

    editor.undo()
    print("After second undo:", editor.get_content())  # Output: (empty string)

    editor.undo()  # Output: Nothing to undo.
    print("Final content:", editor.get_content())  # Output: (empty string)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Current content: Hello, world!
After undo: Hello,
After second undo:
Nothing to undo.
Final content:
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code>
```

| Feature | Data Structure | Justification |
|---|---|---|
| Student Login Authentication | **Hash Table** | Login credentials (username/account ID) must be validated quickly. A hash table provides constant-time average lookup, ensuring fast authentication. |
| Assignment Submission Queue | **Queue** | Submissions are processed in the order they are uploaded, following the First-In-First-Out (FIFO) principle. A queue ensures fairness and sequential handling. |
| Top Performer Ranking | **Priority Queue** | Students must be ranked by scores, with the highest performers accessed first. A priority queue efficiently maintains ranking by priority values. |
| Course Prerequisite Structure | **Graph** | Courses often have dependencies (e.g., Course B requires Course A). A graph models these relationships and supports traversal algorithms to check prerequisites. |
| Undo Feature in Online Editor | **Stack** | Undo operations require reversing the most recent action first. A stack follows the Last-In-First-Out (LIFO) principle, making it ideal for implementing undo functionality. |

**Justification:**

Undo requires reversing the most recent action first, which follows the Last-In-First-Out (LIFO) principle.A stack naturally supports this behavior by storing actions and popping the latest one when undo is triggered.This ensures efficient and accurate reversal of user operations in the editor.

**Task Description #16: Smart Airport Management System – Data Structure Challenge**

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.

2. Emergency Landing Priority – Aircraft in emergency get priority.

3. Flight Information Lookup – Fast search using Flight ID.

4. Air Route Connectivity – Airports connected via routes.

5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

• Choose suitable data structures from the provided list.

• Provide 2–3 sentence justification per feature.

• Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

• Mapping table.

• Working Python implementation.

```python
#generate a code to implement Air Route Connectivity using graph dtastructure with comments and docstrings
class Graph:
    """
    A class to represent a graph for air route connectivity.
    Attributes:
    -----------
    graph : dict
        A dictionary to store the adjacency list of the graph.
    Methods:
    --------
    add_edge(source, destination):
        Adds an edge between the source and destination airports.
    display():
        Displays the adjacency list of the graph.
    """
    def __init__(self):
        """Initializes an empty graph."""
        self.graph = {}
    def add_edge(self, source, destination):
        """
        Adds an edge between the source and destination airports.
        Parameters:
        -----------
        source : str
            The source airport code.
        destination : str
            The destination airport code.
        """
        if source not in self.graph:
            self.graph[source] = []
        if destination not in self.graph:
            self.graph[destination] = []
        self.graph[source].append(destination)
        self.graph[destination].append(source)  # Assuming it's a bidirectional route
```

```python
    def display(self):
        """Displays the adjacency list of the graph."""
        for airport, routes in self.graph.items():
            print(f"{airport}: {', '.join(routes)}")
# Example usage
if __name__ == "__main__":
    air_route_graph = Graph()
    # Adding some air routes
    air_route_graph.add_edge("JFK", "LAX")
    air_route_graph.add_edge("JFK", "ORD")
    air_route_graph.add_edge("LAX", "ORD")
    air_route_graph.add_edge("LAX", "DFW")
    air_route_graph.add_edge("ORD", "DFW")
    # Displaying the graph
    print("Air Route Connectivity:")
    air_route_graph.display()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Air Route Connectivity:
JFK: LAX, ORD
LAX: JFK, ORD, DFW
ORD: JFK, LAX, DFW
DFW: LAX, ORD
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code>
```

| Feature | Data Structure | Justification |
|---|---|---|
| Passenger Check-In Queue | **Queue** | Passengers are processed in the order they arrive, following the First-In-First-Out (FIFO) principle. A queue ensures fairness and sequential handling of check-ins. |

| Feature | Data Structure | Justification |
|---|---|---|
| Emergency Landing Priority | **Priority Queue** | Aircraft in emergency situations must be prioritized over regular landings. A priority queue allows assigning higher priority so critical flights are handled first. |
| Flight Information Lookup | **Hash Table** | Flights are identified by unique IDs. A hash table provides constant-time average lookup, ensuring fast retrieval of flight details. |
| Air Route Connectivity | **Graph** | Airports and routes form a network of nodes and edges. A graph models these connections and supports shortest path algorithms for efficient route planning. |
| Boarding Process (Cabin Storage) | **Stack** | Overhead luggage storage follows a Last-In-First-Out (LIFO) principle, as the last bag placed is the first to be removed. A stack is ideal for simulating this process. |

**Justification:**

Airports and flight routes form a network of interconnected nodes and edges, which is naturally modeled by a graph.Graphs allow representation of both direct and indirect connections, supporting efficient traversal across multiple airports.Algorithms like Dijkstra's or BFS can be applied to find shortest paths, optimize routes, and improve overall connectivity management.

**Task Description #17: Smart Social Media Platform – Data Structure Selection**

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.

2. Trending Hashtags Tracker – Rank hashtags by usage frequency.

3. User Profile Search – Quick lookup using username.

4. Friend Network Representation – Represent user connections.

5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

• Select appropriate data structures from the list.

• Justify your choices clearly.

• Implement one selected feature as a Python program using AI- assisted code generation.

Expected Output:

• Table mapping features to data structures with justification.

• Working Python code with docstrings and comments.

```python
#generate a code to implement News Feed Generation using linkedlist datastructure with comments and docstrings
class Node:
    """
    A class to represent a node in the linked list.

    Attributes:
    data (str): The news feed item.
    next (Node): The reference to the next node in the linked list.
    """
    def __init__(self, data):
        """
        Initializes a new node with the given data.
        Parameters:
        data (str): The news feed item to be stored in the node.
        """
        self.data = data
        self.next = None
class NewsFeed:
    """
    A class to represent a news feed using a linked list.
    Attributes:
    head (Node): The reference to the first node in the linked list.
    """
    def __init__(self):
        """
        Initializes an empty news feed.
        """
        self.head = None
    def add_news(self, news_item):
        """
        Adds a news item to the news feed.
        Parameters:
        news_item (str): The news item to be added to the feed.
        """
```

```python
        new_node = Node(news_item)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node

    def display_feed(self):
        """
        Displays all the news items in the news feed.
        """
        current = self.head
        while current:
            print(current.data)
            current = current.next
# Example usage
if __name__ == "__main__":
    news_feed = NewsFeed()
    news_feed.add_news("Breaking News: New Python Version Released!")
    news_feed.add_news("Sports Update: Local Team Wins Championship!")
    news_feed.add_news("Weather Alert: Heavy Rain Expected Tomorrow!")

    print("News Feed:")
    news_feed.display_feed()

```

```
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
News Feed:
Breaking News: New Python Version Released!
Sports Update: Local Team Wins Championship!
Weather Alert: Heavy Rain Expected Tomorrow!
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code>
```

| Feature | Data Structure | Justification |
|---|---|---|
| News Feed Generation | **Linked List** | Posts must be displayed chronologically. A linked list allows efficient sequential storage and traversal, maintaining the order of posts as they are added. |
| Trending Hashtags Tracker | **Priority Queue** | Hashtags should be ranked by usage frequency, with the most popular ones accessed first. A priority queue efficiently maintains this ranking by priority values. |
| User Profile Search | **Hash Table** | Usernames are unique identifiers. A hash table provides constant-time average lookup, ensuring fast retrieval of user profiles. |
| Friend Network Representation | **Graph** | Friend connections form a network of users (nodes) and relationships (edges). A graph models these connections and supports traversal algorithms for mutual friends or recommendations. |
| Message Undo Feature | **Stack** | Undo requires reversing the most recent message sent, which follows the Last-In-First-Out (LIFO) principle. A stack is ideal for implementing this functionality. |

**Justification:**

A news feed must display posts in chronological order, and a linked list naturally preserves sequential data.New posts can be added efficiently at the end without shifting existing elements.Traversal through the list allows smooth display of posts in the exact order they were uploaded.

**Task #18: Smart University Hostel Management**

Scenario:A university wants to build a hostel management system that manages:

1. Room Allotment

2. Medical Emergency Handling

3. Student Record Retrieval

4. Hostel Block Connectivity

5. Visitor Entry Log

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

```python
#Generate code to implement Room Allotment using queue data structure with comments and docstrings
class RoomAllotment:
    """
    A class to represent a room allotment system using a queue data structure.

    Attributes:
    rooms (list): A list to store the available rooms in the queue.
    """

    def __init__(self):
        """Initialize the RoomAllotment with an empty list of rooms."""
        self.rooms = []

    def add_room(self, room_number):
        """
        Add a room to the queue.

        Parameters:
        room_number (str): The number of the room to be added.
        """
        self.rooms.append(room_number)
        print(f"Room {room_number} added to the queue.")

    def allot_room(self):
        """
        Allot a room from the queue.

        Returns:
        str: The number of the allotted room or a message if no rooms are available.
        """
        if not self.rooms:
            return "No rooms available for allotment."

        allotted_room = self.rooms.pop(0)  # Remove and return the first room in the queue
        return f"Room {allotted_room} has been allotted."

```

```python
    def allot_room(self):
        """
        Allot a room from the queue.

        Returns:
        str: The number of the allotted room or a message if no rooms are available.
        """
        if not self.rooms:
            return "No rooms available for allotment."

        allotted_room = self.rooms.pop(0)  # Remove and return the first room in the queue
        return f"Room {allotted_room} has been allotted."

    def display_rooms(self):
        """Display the current rooms in the queue."""
        if not self.rooms:
            print("No rooms available in the queue.")
        else:
            print("Rooms in the queue:", ", ".join(self.rooms))
# Example usage
if __name__ == "__main__":
    room_allotment = RoomAllotment()
    room_allotment.add_room("101")
    room_allotment.add_room("102")
    room_allotment.add_room("103")
    room_allotment.display_rooms()
    print(room_allotment.allot_room())
    print(room_allotment.allot_room())
    room_allotment.display_rooms()
    print(room_allotment.allot_room())
    print(room_allotment.allot_room())
    room_allotment.display_rooms()
```

```
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Room 101 added to the queue.
Room 102 added to the queue.
Room 103 added to the queue.
Rooms in the queue: 101, 102, 103
Room 101 has been allotted.
Room 102 has been allotted.
Rooms in the queue: 103
Room 103 has been allotted.
No rooms available for allotment.
No rooms available in the queue.
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code>
```

| Feature | Data Structure | Justification |
|---|---|---|
| Room Allotment | **Queue** | Rooms are allotted to students in the order they apply, following the First-In-First-Out (FIFO) principle. A queue ensures fairness and sequential allocation. |
| Medical Emergency Handling | **Priority Queue** | Emergency cases must be prioritized over regular requests. A priority queue allows assigning higher priority so critical medical cases are handled first. |
| Student Record Retrieval | **Hash Table** | Each student has a unique ID or roll number. A hash table provides constant-time average lookup, ensuring fast retrieval of student records. |
| Hostel Block Connectivity | **Graph** | Hostel blocks and rooms form a network of nodes and edges. A graph models these connections and supports traversal algorithms for navigation and connectivity. |

| Feature | Data Structure | Justification |
|---|---|---|
| Visitor Entry Log | **Linked List** | Visitor entries must be maintained in chronological order. A linked list allows efficient sequential storage and traversal of visitor logs. |

**Justification:**

Room allotment follows a First-In-First-Out (FIFO) principle, where students are assigned rooms in the order they apply.A queue ensures fairness by maintaining the sequence of requests without skipping anyone.This structure avoids conflicts and guarantees systematic allocation of hostel rooms.

**Task #19: Smart Electricity Distribution System**

Scenario: An electricity board plans to develop a system that includes:

1. Complaint Handling

2. Fault Priority Handling

3. Consumer Record Lookup

4. Power Grid Connectivity

5. Meter Reading History Tracking

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings.

```python
#generate a code to implement Fault Priority Handling using priority queue data structure with comments and docstrings
import heapq
class Fault:
    """
    A class to represent a fault in the system.

    Attributes:
    -----------
    name : str
        The name of the fault.
    priority : int
        The priority of the fault (lower number means higher priority).
    """

    def __init__(self, name, priority):
        """
        Initializes a new Fault instance.

        Parameters:
        -----------
        name : str
            The name of the fault.
        priority : int
            The priority of the fault (lower number means higher priority).
        """
        self.name = name
        self.priority = priority

```

```python
    def __lt__(self, other):
        """
        Defines the less than operator for Fault objects based on their priority.

        Parameters:
        -----------
        other : Fault
            Another Fault object to compare with.

        Returns:
        --------
        bool
            True if this Fault has a higher priority (lower number) than the other, False otherwise.
        """
        return self.priority < other.priority
class FaultPriorityQueue:
    """
    A class to represent a priority queue for handling faults.

    Attributes:
    -----------
    queue : list
        A list to store the faults in the priority queue.
    """

    def __init__(self):
        """
        Initializes a new FaultPriorityQueue instance.
        """
        self.queue = []

```

```python
    def add_fault(self, fault):
        """
        Adds a fault to the priority queue.
        Parameters:
        -----------
        fault : Fault
            The fault to be added to the queue.
        """
        heapq.heappush(self.queue, fault)

    def handle_fault(self):
        """
        Handles the highest priority fault in the queue.
        Returns:
        Fault
            The fault that was handled, or None if the queue is empty.
        """
        if not self.queue:
            return None
        return heapq.heappop(self.queue)
# Example usage
if __name__ == "__main__":
    # Create a fault priority queue
    fault_queue = FaultPriorityQueue()
    # Add some faults to the queue
    fault_queue.add_fault(Fault("Fault A", 2))
    fault_queue.add_fault(Fault("Fault B", 1))
    fault_queue.add_fault(Fault("Fault C", 3))
    # Handle faults in order of priority
    while True:
        fault = fault_queue.handle_fault()
        if fault is None:
            print("No more faults to handle.")
            break
        print(f"Handling {fault.name} with priority {fault.priority}.")
```

```
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_11.py
Handling Fault B with priority 1.
Handling Fault A with priority 2.
Handling Fault C with priority 3.
No more faults to handle.
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code>
```

| Feature | Data Structure | Justification |
|---|---|---|
| Complaint Handling | **Queue** | Complaints must be addressed in the order they are received, following the First-In-First-Out (FIFO) principle. A queue ensures fairness and systematic resolution. |
| Fault Priority Handling | **Priority Queue** | Faults vary in severity, and critical faults must be resolved first. A priority queue allows assigning higher priority to urgent faults, ensuring efficient handling. |
| Consumer Record Lookup | **Hash Table** | Each consumer has a unique ID or account number. A hash table provides constant-time average lookup, enabling fast retrieval of consumer records. |
| Power Grid Connectivity | **Graph** | Power grids consist of interconnected substations and transmission lines. A graph models these nodes and edges, supporting traversal and shortest path algorithms for efficient distribution. |

| Feature | Data Structure | Justification |
|---|---|---|
| Meter Reading History Tracking | **Linked List** | Meter readings are recorded chronologically. A linked list maintains sequential data efficiently, allowing easy traversal of past readings in order. |

**Justification:**

Faults in electricity distribution vary in severity, and critical faults must be resolved before minor ones.A priority queue allows assigning urgency levels so the most critical faults are always handled first.This ensures efficient resource allocation and minimizes downtime by prioritizing urgent repairs.